

OpenSSL (KF)

Login, jméno a příjmení řešitele.

FIC0024, Richard Ficek

1. **Symetrická kryptografie** - Který algoritmus a jaké režimy jste použili? Jaké příkazy a parametry jste použili?

Pro symetrickou kryptografii jsem použil algoritmus AES-128 v režimech CBC (Cipher Block Chaining) a ECB (Electronic Codebook).

CBC

```
openssl enc -aes-128-cbc -salt -in text.txt -out text_cbc.enc
```

```
openssl enc -d -aes-128-cbc -in text_cbc.enc -out text_decrypted.txt
```

ECB

```
openssl enc -aes-128-ecb -in text.txt -out text_ecb.enc
```

```
openssl enc -d -aes-128-ecb -in text_ecb.enc -out text_ecb_dec.txt
```

2. **Symetrická kryptografie – režimy**

- a) Kolik informace (tj. kolik otevřeného textu) bylo po dešifrování změněného šifrovaného textu (textu se změněným bitem (bajtem)) porušeno, pokud byly použity režimy ECB resp. CBC? Vysvětlete prosím, proč.

- b) Co se stalo, když jste místo změny hodnoty jednoho bitu(bajtu) jeden bit (bajt) z šifrovaného textu úplně odstranili a pokusili se porušený šifrový text dešifrovat?

3. **Hašovací funkce - generování haše:-** Jaké příkazy a parametry jste použili? Jsou haše stejné? Jsou haše stejně dlouhé? A jak jsou jednotlivé haše dlouhé?

Použité příkazy:

```
openssl dgst -md5 text.txt, openssl dgst -sha1 text.txt, openssl dgst -sha256 text.txt
```

Výsledné haše:

```
MD5(text.txt)= 88b940907b77b3af3f85ec779b77415d
```

```
SHA1(text.txt)= 1ad098a538e0f4381e1608c8a8bb62b06dc6d463
```

```
SHA256(text.txt)= 1de8c2a081b42648be2096a8bc23a32bde4b3fd7d43090b40fc92861ee0cdf
```

Haše nejsou stejné a mají různé délky:

- MD5: 32 hex znaků (128 bitů)

- SHA1: 40 hex znaků (160 bitů)

- SHA256: 64 hex znaků (256 bitů)

4. Hašovací funkce - náhodnost jednocestné hašovací funkce

a) Jsou haše H_1 a H_2 stejné nebo ne?

Ne, nejsou stejné. I změna jednoho bajtu vstupního souboru způsobila úplně odlišný hash.

b) Kolik stejných bitů (bitů na dané pozici) mají haše H_1 a H_2 ?

125

c) Popište, jakým způsobem jste to určili (jaký příkaz jste použili nebo jakou aplikaci jste použili (vložte zdrojový kód), nebo např. jaké webové stránky). Nezajímá mne počet znaků, ale bitů.

```
h1="1de8c2a081b42648be2096a8bc23a32bde4b3fd7d43090b40fc92861ee0cdf"
h2="976c90c09ee072c2ed946db4d82a5cc2968c82225a7125447bddae612e2491ea"
b1 = bin(int(h1,16))[2:].zfill(256)
b2 = bin(int(h2,16))[2:].zfill(256)
same_bits = sum(bit1==bit2 for bit1,bit2 in zip(b1,b2))
print(same_bits)
```

5. Asymetrická kryptografie - generování klíčů

a) Jaká metoda, resp. jaký algoritmus byl použit pro generování klíčů?

Byla použita asymetrická kryptografie RSA (Rivest – Shamir – Adleman)
Délka klíče je 1024 bitů

b) Jakou hlavičku a patičku má tento soubor, co z ní vyčteme?

Hlavička: -----BEGIN RSA PRIVATE KEY-----

Patička: -----END RSA PRIVATE KEY-----

Z hlavičky a patičky víme, že soubor obsahuje privátní klíč formátu PEM a algoritmu RSA

c) Jakou hodnotu má veřejný klíč?

```
modulus: 00:c0:af:f2:b1:3d:bd:c8:42:61:75:66:77:c2:cc: d4:57:db:9d:eb:d3:07:88:58:8c:aa:09:a4:a3:b1: 8b:c7:f3:a8:20:a3:d1:94:c1:90:36:0b:
92:26:b6: a5:ae:c2:a0:e0:3a:13:20:e7:08:55:e3:93:eb:e6: 5a:d2:b2:3b:65:bf:41:e4:04:6c:ba:02:79:7d:33: 8e:77:c6:42:0a:bb:3b:69:8e:1c:51:bf:
4c:0c:8a: f4:b2:51:4f:2b:e4:2e:da:fb:0f:bd:9c:8d:8f:85: 00:f5:ed:1d:87:af:0d:1a:fb:51:da:02:f5:59:5a: 98:cc:c2:8f:8e:b9:55:80:13
publicExponent: 65537 (0x10001)
```

d) Co vyčteme z hlavičky a patičky?

Hlavička -----BEGIN PUBLIC KEY----- a patička -----END PUBLIC KEY----- označují veřejný klíč v PEM formátu
Z hlavičky/patičky nevyčteme hodnoty klíče, jen typ souboru a formát (PEM, veřejný klíč)

6. Asymetrická kryptografie - digitální podpis

a) Jaké příkazy a parametry jste použili?

```
openssl genrsa -out private.pem 1024 generuje privátní klíč
openssl rsa -in private.pem -pubout -out public.pem export veřejného klíče
openssl dgst -sha256 -sign private.pem -out F1.sha256 text.txt vytvoření podpisu
openssl dgst -sha256 -verify public.pem -signature F1.sha256 text.txt ověření podpisu
```

b) Podařilo se Vám ověřit "originální" podpis?

Ano, pokud soubor text.txt nebyl změněn, výstup byl Verified OK.

c) Podařilo se Vám ověřit podpis po změně podepsané haše F1.sha256 a pokud, tak ne proč? Vysvětlete.

Ne, ověření selže, protože podpis je digitálně vázaný na hash souboru.
I jediný bajt změněný v podpisu nebo souboru způsobí, že ověření selže.
Toto zajišťuje integritu dat – podpis potvrzuje, že obsah nebyl změněn.

7. **Asymetrická kryptografie – certifikáty:** Jaký příkaz a parametry jste použili pro ověření certifikátu subjektu PB?

openssl verify -CAfile CA.0.crt PB.0.crt